

PROYECTO - PREDICCIÓN DE INCUMPLIMIENTO DE CRÉDITOS

Descripción general del proyecto

Has iniciado tu labor en el equipo de Datos y Analítica de una empresa financiera, desempeñándote como Científico de Datos Junior Advanced. Tu primera asignación consiste en desarrollar un modelo predictivo mediante técnicas de aprendizaje automático, utilizando información histórica de créditos, con el objetivo de anticipar el comportamiento de nuevos usuarios.

La empresa opera bajo un esquema estructurado de proyectos, en el cual cada iniciativa debe seguir una arquitectura de carpetas estrictamente definida. Esta estructura no puede ser modificada, ya que los procesos de despliegue a producción están automatizados a través de pipelines de validación en Jenkins. Cualquier alteración en la organización de carpetas podría generar retrasos significativos en el paso a producción.

La variable objetivo utilizada en el proyecto es Pago_atiempo. La clasificación utilizada por los scripts interpreta la clase 0 como posible incumplimiento y la clase 1 como pago a tiempo. El problema presenta una distribución de clases desbalanceada, por lo que el análisis no debe depender únicamente de Accuracy.

Flujo general

Etapas	Script principal	Entrada principal	Resultado principal
Preparación y calidad	Cargar_datos.py	Base_de_datos.csv	Base_de_datos_preparada.csv
EDA	Comprension_eda.py	Base_de_datos_preparada.csv	Reporte EDA, CSV y 10 gráficos
Feature Engineering	ft_engineering.py	Base_de_datos_preparada.csv	Train/test transformados y pipeline
Entrenamiento	model_training_evaluation.py	Train/test transformados	Modelos evaluados y modelo_final.joblib
Deployment	model_deploy.py	Modelo + pipeline + datos	Predicciones y reportes
Servicio	API.py	Modelo + pipeline	Endpoints de predicción
Envío a API	enviar_predicciones_API.py	Base_de_datos.csv	predicciones_api_monitoring.csv

Monitoring	model_monitoring.py	Base preparada + predicciones API	Métricas, drift y análisis temporal
Dashboard	APP.py	Resultados de monitoring	Interfaz Streamlit

La carpeta raíz del proyecto puede ubicarse en cualquier ruta del computador. Lo que debe mantenerse es la estructura interna y la relación entre los archivos. Los scripts construyen sus rutas a partir de la ubicación del propio archivo o de rutas relativas; por eso mover un script, cambiar nombres de artefactos o ejecutarlo desde una estructura diferente puede provocar errores de archivos no encontrados.

Datos de entrada y preparación

La base original es `Base\_de\_datos.csv`. En la versión entregada contiene 10.763 registros y 23 columnas. La variable objetivo `Pago\_atiempo` contiene 10.252 registros de clase 1 y 511 de clase 0, equivalentes aproximadamente a 95,25% y 4,75%, respectivamente.

Esta distribución es relevante desde el inicio porque el objetivo está desbalanceado. Si se observara únicamente el porcentaje global de aciertos, un modelo podría parecer correcto aunque tuviera un comportamiento deficiente al identificar la clase minoritaria. Por eso el proyecto conserva esta característica durante el análisis y posteriormente utiliza métricas por clase y métricas balanceadas.

¿Qué hace Cargar_datos.py?

El primer script funciona como una barrera de control de calidad. Lee `Base\_de\_datos.csv`, revisa dimensiones, tipos, valores nulos y duplicados y realiza validaciones específicas sobre fechas, edades, variables financieras, puntajes, saldos, categorías y objetivo. Cuando encuentra valores claramente inválidos, los convierte en valores nulos en lugar de inventar un valor de reemplazo.

Entre las comprobaciones se encuentran edades fuera del rango 18–100, valores negativos en variables donde no son válidos, salarios menores o iguales a cero, cuotas superiores al salario, puntajes negativos, saldos inconsistentes, categorías de `tendencia\_ingresos` fuera de las tres categorías esperadas y valores inválidos de `Pago\_atiempo`. También se calculan relaciones financieras y se revisan valores extremos mediante el criterio del rango intercuartílico.

El script crea además variables derivadas iniciales: año, mes, día de la semana y hora del préstamo; relación cuota/salario; relación deuda/salario; y saldo pendiente estimado. Los infinitos se convierten a nulos. Es importante notar que esta etapa no elimina masivamente los registros: conserva la estructura de la base y deja los problemas detectados preparados para que las siguientes etapas los traten de manera apropiada.

En la base proporcionada se identificaron inicialmente 7.175 valores nulos y 0 filas completamente duplicadas. La preparación conserva 10.763 registros. La revisión de la base también encontró 150 edades fuera del rango definido, 135 puntajes negativos, 1 puntaje Datacrédito negativo, 24 salarios menores o iguales a cero, 11 cuotas superiores al salario y 58 valores de tendencia fuera de las categorías válidas. Estos valores son convertidos o tratados según las reglas del script.

El resultado directo de esta etapa es ``Base_de_datos_preparada.csv`` y el resumen ``Observaciones_calidad_datos.csv``.

¿Por qué se hace antes del EDA?

El objetivo es evitar que el análisis exploratorio mezcle datos originales con errores que ya sabemos identificar. De esta forma, EDA recibe una versión consistente de la base y puede concentrarse en describir distribuciones, relaciones y patrones, en lugar de volver a realizar controles básicos de integridad.

Análisis exploratorio de datos (EDA)

``Comprension_eda.py`` toma ``Base_de_datos_preparada.csv`` y construye un análisis exploratorio reproducible. Al comenzar, elimina la carpeta anterior ``resultados_eda_entrega`` y la vuelve a crear, de modo que los resultados de una ejecución no se mezclen con archivos antiguos.

El EDA no busca todavía entrenar el modelo. Su función es entender la estructura de los datos, describir variables numéricas y categóricas, revisar el objetivo, estudiar relaciones con ``Pago_atiempo``, observar correlaciones, detectar comportamientos atípicos y revisar la dimensión temporal. La información obtenida aquí ayuda a tomar decisiones posteriores de modelado.

Qué analiza

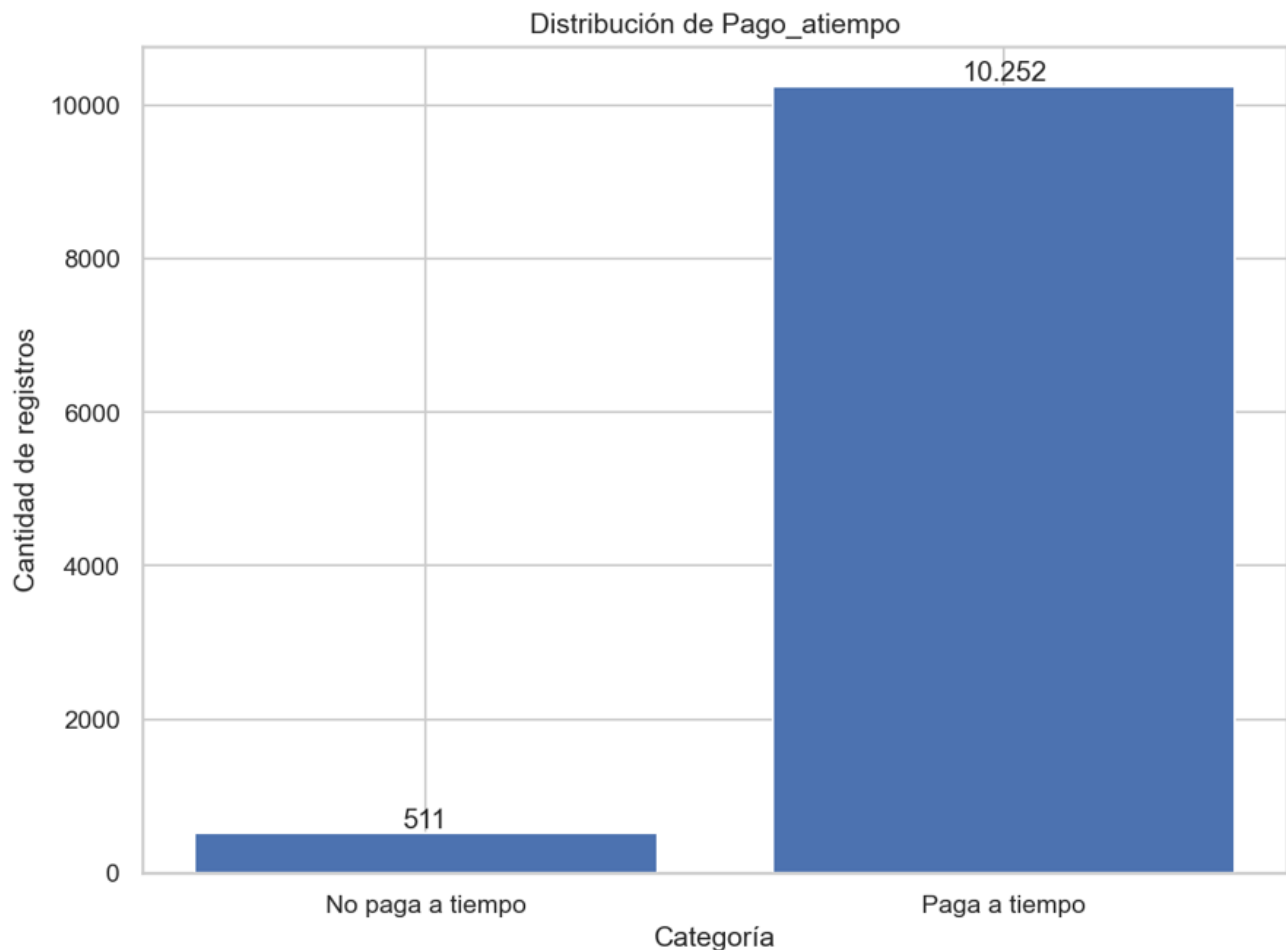
- Resumen general: registros, columnas, celdas, nulos y clasificación de variables.
- Calidad de datos por columna y distribución de la variable objetivo.
- Estadísticas descriptivas y frecuencias de las variables categóricas.
- Comparación de variables numéricas y categóricas según ``Pago_atiempo``.
- Correlaciones entre variables numéricas y correlación de las variables con el objetivo.
- Valores extremos y comportamiento temporal de los préstamos.

Gráficos generados

El script genera exactamente diez gráficos en ``resultados_eda_entrega/graficos/``. Estos son los nombres reales definidos por el código:

- 01_distribuciones_numericas.png
- 02_boxplots_numericos.png
- 03_distribucion_target.png
- 04_frecuencias_categoricas.png

- 05_numericas_por_target.png
- 06_categoricas_por_target.png
- 07_matriz_correlacion.png
- 08_relaciones_financieras.png
- 09_comportamiento_puntajes.png
- 10_comportamiento_temporal.png



Este es el gráfico más directo para mostrar el desbalance del objetivo: 10.252 registros de pago a tiempo frente a 511 registros de no pago a tiempo. La gráfica muestra cantidades, no probabilidades de futuras predicciones.

Además de los gráficos, el EDA genera `Reporte\_EDA.xlsx`, `Reporte\_EDA.txt` y archivos CSV como `resumen\_inicial.csv`, `resumen\_outliers.csv`, `resumen\_temporal.csv`, `calidad\_datos.csv`, `clasificacion\_variables.csv`, `distribucion\_target.csv`, `estadisticas\_numericas.csv`, `frecuencias\_categoricas.csv`, `comparacion\_numerica\_target.csv`, `comportamiento\_categoricas\_target.csv`, `correlaciones.csv`, `correlaciones\_target.csv`, `hallazgos.csv` y `matriz\_correlacion.csv`.

Hallazgos que pasan a la siguiente etapa

El EDA confirma que el objetivo está fuertemente desbalanceado y que existen valores faltantes y distribuciones financieras con comportamientos que deben tratarse durante el preprocesamiento. También permite identificar variables que requieren cuidado metodológico. En particular, `puntaje` se considera posteriormente una variable potencialmente problemática por posible fuga de información y se excluye del modelado.

La correlación, una diferencia entre grupos o una variable con gran importancia posterior no debe interpretarse automáticamente como causalidad. El EDA sirve para describir relaciones observadas y apoyar decisiones técnicas; no demuestra por sí mismo que una variable provoque el incumplimiento.

Ingeniería de características y preprocesamiento

`ft\_engineering.py` toma la base preparada y convierte la información en una representación adecuada para entrenar los modelos. Esta etapa es importante porque el modelo no debe recibir directamente una mezcla de fechas, texto, valores faltantes y escalas financieras diferentes.

Creación de características

La fecha del préstamo se transforma en variables de año, mes, día, día de la semana, semana del año, trimestre, hora y una variable binaria de fin de semana. La fecha original se elimina después de extraer estas características.

También se construyen `relacion\_cuota\_salario`, `relacion\_deuda\_salario` y `saldo\_pendiente\_estimado`. Estas variables intentan representar relaciones financieras más informativas que una cifra aislada. `tipo\_credito` se convierte a variable categórica.

Separación y prevención de fuga

La variable `Pago\_atiempo` se separa como objetivo. `puntaje` se excluye explícitamente por su posible fuga de información. La intención es evitar que el modelo aprenda a partir de una variable que podría incorporar información relacionada con el resultado que se intenta predecir.

División train/test

El código utiliza `train\_test\_split` con 80% de los registros para entrenamiento y 20% para prueba, `random\_state=42` y `stratify=y`. Con 10.763 registros, esto corresponde a 8.610 registros de entrenamiento y 2.153 de prueba.

Preprocesamiento

Las variables numéricas pasan por imputación mediante mediana y estandarización con `StandardScaler`. Las variables categóricas pasan por imputación con la categoría más frecuente y `OneHotEncoder(handle\_unknown="ignore")`. El preprocesador se ajusta únicamente con entrenamiento y después se aplica al conjunto de prueba. Esto evita que información del conjunto de prueba influya en el aprendizaje del preprocesamiento.

En la base preparada, después de separar objetivo, `puntaje` y fecha, quedan 27 variables predictoras: 24 numéricas y 3 categóricas. Con la codificación One-Hot de las categorías presentes, la representación transformada llega a 35 características.

Los resultados se guardan en `resultados\_feature\_engineering`:
`X\_train\_transformado.csv`, `X\_test\_transformado.csv`, `y\_train.csv`, `y\_test.csv`,
`pipeline\_preprocesamiento.joblib`, `nombres\_caracteristicas.csv`,
`resumen\_valores\_nulos.csv`, `resumen\_variables.csv` y
`resumen\_feature\_engineering.txt`.

Entrenamiento y evaluación de modelos

El entrenamiento recibe los conjuntos ya transformados. La implementación actual utiliza tres algoritmos: Regresión Logística, Random Forest y SVM calibrado.

Modelo	Configuración documentada en el código	Función en el proyecto
Regresión Logística	<code>class_weight='balanced', max_iter=2000</code>	Modelo de referencia para comparar el comportamiento de una técnica lineal.
Random Forest	300 árboles, <code>class_weight='balanced'</code>	Modelo basado en conjunto de árboles para comparar un enfoque no lineal.
SVM calibrado	SVC balanceado + CalibratedClassifierCV, método sigmoid, cv=3	Modelo de clasificación adicional para comparar otra forma de separación de clases.

La configuración `class_weight="balanced"` se utiliza en los modelos que la soportan para considerar el desbalance de clases durante el entrenamiento. La SVM base se envuelve en `CalibratedClassifierCV`, lo que permite obtener probabilidades para las evaluaciones que requieren `predict_proba`.

¿Por qué se prueban varios modelos?

No se parte de la idea de que una técnica específica será adecuada de antemano. Los tres modelos se entrenan sobre los mismos datos y se evalúan con una estructura común. Esto permite observar diferencias de comportamiento y seleccionar el modelo final con criterios relacionados con el objetivo del proyecto: detectar registros de la clase 0, que representa posibles incumplimientos.

Métricas utilizadas

El código calcula Accuracy, Balanced Accuracy, ROC-AUC y PR-AUC, además de Precision, Recall y F1-score para las clases 0 y 1 y las métricas macro. También genera matrices de confusión para los modelos entrenados.

La validación cruzada utiliza cinco particiones y `F1-score macro` como métrica. Esto permite observar el comportamiento del modelo en diferentes particiones del entrenamiento en lugar de depender de una sola división.

Selección del modelo final

La selección no se basa únicamente en Accuracy. El código ordena los modelos por Recall de la clase 0, después por F1-score de la clase 0 y finalmente por Balanced Accuracy. El primer modelo según ese orden se guarda como `modelo\_final.joblib` tanto dentro de la carpeta de resultados como en la carpeta raíz del proyecto.

Este criterio refleja una decisión metodológica concreta: para este problema interesa especialmente no perder la capacidad de identificar los registros clasificados como `No paga a tiempo`. Las métricas adicionales siguen siendo importantes para entender el comportamiento global y el equilibrio entre ambas clases.

Model Deployment

El script vuelve a crear las mismas variables derivadas necesarias para el modelo, separa `Pago\_atiempo` cuando está disponible, elimina `puntaje` y la fecha original, transforma los datos utilizando el pipeline guardado y verifica que la cantidad de características resultante coincida con lo que espera el modelo.

Después genera la predicción, obtiene las probabilidades de las clases cuando el modelo las permite y crea una interpretación legible: clase 0 como `Posible incumplimiento` y clase 1 como `Pago a tiempo`. Si existen valores reales de `Pago\_atiempo`, también calcula Accuracy, Balanced Accuracy, Precision, Recall y F1 por clase y una matriz de confusión.

API de predicción con FastAPI

`API.py` convierte el modelo en un servicio consumible mediante solicitudes HTTP. Al iniciar, carga `modelo\_final.joblib` y

`resultados\_feature\_engineering/pipeline\_preprocesamiento.joblib`. La API define el formato esperado para un registro de crédito y el formato de respuesta.

Antes de predecir, la API vuelve a crear las variables derivadas utilizadas durante el feature engineering: variables temporales, relación cuota/salario, relación deuda/salario y saldo pendiente estimado. Después aplica el mismo pipeline y ejecuta el modelo. Esto es importante porque el servicio debe reproducir las transformaciones utilizadas durante el entrenamiento.

Respuesta de la API

Cada predicción devuelve un identificador UUID, la clase predicha, el resultado legible, la probabilidad de incumplimiento, la probabilidad de pago a tiempo, el nivel de riesgo y el tiempo de predicción en milisegundos. El UUID identifica la predicción; no es un mecanismo de cifrado.

El nivel de riesgo se determina con la probabilidad de incumplimiento: menor de 0,40 corresponde a `Bajo`, desde 0,40 y menor de 0,70 a `Medio`, y 0,70 o más a `Alto`.

Endpoints

Endpoint	Método	Función
/	GET	Confirma que la API está funcionando y muestra los endpoints disponibles.
/health	GET	Comprueba que la API, el modelo y el pipeline estén cargados.
/predict	POST	Procesa un solo registro.
/predict/batch	POST	Procesa una lista de registros.

La documentación interactiva queda disponible en `/docs`. El endpoint batch es el que utiliza el script `enviar\_predicciones\_API.py`.

Persistencia de predicciones

Cada predicción se agrega a `predicciones\_api\_monitoring.csv`. Esto conecta directamente la API con la etapa de monitoreo: la API produce las predicciones y el monitoring utiliza ese archivo como fuente de datos actuales.

Envío de registros a la API

`enviar\_predicciones\_API.py` simula la llegada de nuevos registros al servicio. Lee `Base\_de\_datos.csv`, comprueba que existan las columnas requeridas, selecciona hasta 100 registros mediante `random\_state=42`, convierte los tipos y completa valores faltantes antes de construir la solicitud.

Los valores numéricos faltantes se completan con la mediana de la muestra seleccionada y las categorías faltantes se reemplazan por `Desconocido`. La fecha se convierte a un formato definido y las columnas que deben ser enteras se redondean y convierten.

El script envía los registros al endpoint `/predict/batch`. Por eso la API debe estar ejecutándose antes de lanzar este script. La respuesta recibida se guarda como `predicciones\_api\_monitoring.csv`.

La API debe permanecer activa en una terminal mientras el script de envío se ejecuta desde una segunda terminal.

Terminal 1: `uvicorn API:app --host 127.0.0.1 --port 8000`

Terminal 2: `python enviar_predicciones_API.py`

La separación en dos terminales es intencional: una mantiene el servicio disponible y la otra actúa como cliente que envía los registros.

Model Monitoring

`model\_monitoring.py` compara una base de referencia con las predicciones recientes generadas por la API. La referencia es `Base\_de\_datos\_preparada.csv` y la muestra actual es `predicciones\_api\_monitoring.csv`.

Antes de calcular métricas, el script valida que las predicciones sean 0 o 1, que las probabilidades estén entre 0 y 1 y que la suma de las probabilidades de incumplimiento y pago a tiempo esté aproximadamente entre 0,99 y 1,01. Solo las predicciones que pasan estas comprobaciones se consideran válidas para el monitoreo.

Qué se monitorea

- Distribución de las clases predichas.
- Probabilidad de incumplimiento: promedio, mediana, mínimo y máximo.
- Niveles de riesgo y porcentaje de riesgo alto.
- Tiempo de predicción.
- Cambios en las distribuciones de las variables respecto a la referencia.
- Comportamiento temporal de las predicciones.

Data Drift

El análisis de drift compara las variables de referencia con las observadas en la muestra actual. Para variables numéricas se utilizan Kolmogorov-Smirnov, PSI y Jensen-Shannon; para variables categóricas se utilizan PSI, Jensen-Shannon y Chi-cuadrado. Las reglas de detección del script consideran, entre otros criterios, p-valores inferiores a 0,05, PSI de al menos 0,25 y Jensen-Shannon de al menos 0,10, según el tipo de variable y la métrica.

En la ejecución documentada, realizada sobre una muestra actual de 101 registros, se evaluaron 20 variables y 15 fueron marcadas con drift, equivalente al 75% de las variables evaluadas. Este resultado corresponde a esta muestra y ejecución específica; no debe interpretarse como una condición permanente del modelo o de todos los datos futuros. Esta cifra significa `15 de 20 variables`, no que 75% de los clientes hayan cambiado ni que 75% de los clientes tengan riesgo.

Entre los resultados actuales destacan `tendencia\_ingresos`, `promedio\_ingresos\_datacredito`, `cuota\_pactada` y `tipo\_credito` con señales de drift según las métricas calculadas. El archivo `data\_drift\_resultados.csv` conserva los estadísticos y banderas utilizados para estas conclusiones.

Salidas

El script genera `metricas\_monitoring.csv`, `distribucion\_predicciones.csv`, `predicciones\_validas\_monitoring.csv`, `resumen\_niveles\_riesgo.csv`, `alertas\_monitoring.txt`, `data\_drift\_resultados.csv`, `data\_drift\_alertas.txt`, `variables\_con\_drift.csv`, gráficos de distribución, gráficos de drift dentro de `data\_drift\_graficos/` y `analisis\_temporal.csv` junto con `analisis\_temporal\_predicciones.png`.

Resultado observado en la muestra actual

En el archivo de predicciones API proporcionado se observan 101 registros. El modelo clasificó 53 como `Posible incumplimiento` y 48 como `Pago a tiempo`, equivalentes a 52,48% y 47,52%. La probabilidad media de incumplimiento fue 51,92%, con una mediana de 51,12%, mínimo de 20,13% y máximo de 100%. Los niveles de riesgo fueron 24 bajos, 65 medios y 12 altos. El tiempo medio de predicción fue 27,21 ms.

Estos porcentajes corresponden a las predicciones del modelo sobre la muestra procesada por la API. No deben confundirse con la distribución histórica del dataset original, que contiene 95,25% de pagos a tiempo y 4,75% de no pagos a tiempo. Tampoco significan que el 52,48% de esos clientes efectivamente incumplirá: son clasificaciones del modelo sobre una muestra nueva.

Aplicación de Streamlit

`APP.py` construye una interfaz para consultar los resultados del monitoring. La aplicación no vuelve a entrenar el modelo; lee los archivos generados por el proceso de monitoreo y los presenta de forma visual.

La aplicación tiene cuatro secciones: Resumen, Predicciones, Data Drift y Análisis temporal. En Resumen muestra el total de predicciones, porcentaje de incumplimiento, probabilidad promedio, cantidad de riesgo alto y tiempo promedio. También presenta tres gráficos: distribución de predicciones, distribución de probabilidad de incumplimiento y distribución de niveles de riesgo.

La sección Predicciones permite filtrar por nivel de riesgo y por rango de probabilidad de incumplimiento. Data Drift muestra las variables monitorizadas, las variables con drift, la tabla completa de resultados y los gráficos almacenados en `data\_drift\_graficos`. Análisis temporal presenta la tabla temporal y el gráfico `analisis\_temporal\_predicciones.png`.

Por tanto, Streamlit funciona como capa de visualización del monitoring: no sustituye los scripts anteriores, sino que consume sus resultados.

Ejecución y reproducibilidad

Regla fundamental de ejecución

Todos los scripts deben conservarse en la carpeta principal del proyecto junto con los archivos y carpetas que necesitan. Deben ejecutarse desde esa carpeta. Los scripts usan rutas construidas a partir de la ubicación del archivo o rutas relativas, por lo que moverlos, cambiar los nombres de los artefactos o ejecutarlos desde otra ubicación puede provocar errores de archivos no encontrados.

La ruta absoluta de la carpeta puede variar según el equipo de cada usuario. No es necesario utilizar una ruta personal específica; lo importante es conservar la estructura interna del proyecto y ejecutar los comandos desde la carpeta raíz.

Instalación y preparación del entorno

Antes de ejecutar el flujo del proyecto se deben instalar las dependencias definidas en requirements.txt. Se recomienda utilizar un entorno virtual para mantener aisladas las librerías del proyecto.

En Windows, desde la carpeta raíz del proyecto:

```
python -m venv venv
```

```
venv\Scripts\activate
```

```
pip install -r requirements.txt
```

Una vez instaladas las dependencias, los scripts pueden ejecutarse siguiendo el orden establecido en este documento.

Orden recomendado

1. Ejecutar `Cargar\_datos.py` para generar `Base\_de\_datos\_preparada.csv` y el informe de calidad.
2. Ejecutar `Comprension\_eda.py` para regenerar el EDA, sus tablas y sus 10 gráficos.
3. Ejecutar `ft\_engineering.py` para generar los conjuntos transformados y el pipeline.
4. Ejecutar `model\_training\_evaluation.py` para entrenar los tres modelos, comparar sus métricas y guardar `modelo\_final.joblib`.
5. Ejecutar `model\_deploy.py` para aplicar el modelo final y generar las predicciones y reportes de deployment.
6. Iniciar la API mediante uvicorn para disponer del servicio de predicción:

```
uvicorn API:app --host 127.0.0.1 --port 8000
```

7. Manteniendo la API abierta, ejecutar ``enviar_predicciones_API.py`` para enviar hasta 100 registros al endpoint batch.
8. Ejecutar ``model_monitoring.py`` para validar y analizar las predicciones producidas por la API.
9. Ejecutar `APP.py` mediante Streamlit para visualizar los resultados del monitoring:
`python -m streamlit run APP.py`

Ejecución de Streamlit

```
python -m streamlit run APP.py
```

Ejecución de la API

```
uvicorn API:app --host 127.0.0.1 --port 8000
```

La API puede detenerse cerrando la terminal donde está ejecutándose o interrumpiendo el proceso. No es necesario mantenerla abierta cuando no se estén realizando predicciones. Sí debe estar activa mientras ``enviar_predicciones_API.py`` envía registros.

Algunos artefactos son fundamentales para continuar el flujo.

``Base_de_datos_preparada.csv`` alimenta EDA y feature engineering;

``pipeline_preprocesamiento.joblib`` acompaña al modelo para transformar nuevos datos;

``modelo_final.joblib`` es el artefacto que utiliza deployment y API; y

``predicciones_api_monitoring.csv`` conecta la API con monitoring y Streamlit.

Contenerización con Docker

El proyecto incorpora Docker como mecanismo de empaquetado y ejecución reproducible del servicio de predicción. La contenerización se utiliza específicamente para desplegar la API desarrollada con FastAPI junto con las dependencias y artefactos necesarios para realizar inferencias.

El archivo Dockerfile define la configuración de la imagen. El proceso utiliza `python:3.13-slim` como imagen base y establece `/app` como directorio de trabajo dentro del contenedor. Posteriormente, copia `requirements.txt` e instala las dependencias necesarias mediante `pip`.

Después de instalar las dependencias, el Dockerfile incorpora los archivos necesarios para el servicio:

`API.py`

`modelo_final.joblib`

`resultados_feature_engineering/pipeline_preprocesamiento.joblib`

Finalmente, se expone el puerto 8000 y se configura Uvicorn para iniciar automáticamente la aplicación FastAPI cuando se ejecuta el contenedor.

Construcción de la imagen

La construcción debe realizarse desde la carpeta raíz del proyecto, donde se encuentran el Dockerfile, requirements.txt, el modelo y los demás archivos requeridos:

```
docker build -t modelo-creditos-api .
```

Durante este proceso Docker utiliza el Dockerfile para crear una imagen que contiene el entorno de ejecución, las dependencias de Python, la API y los artefactos necesarios para realizar las predicciones.

Ejecución del contenedor

Una vez construida la imagen, se inicia un contenedor mediante:

```
docker run -d -p 8000:8000 --name modelo-creditos-api modelo-creditos-api
```

El parámetro -d ejecuta el contenedor en segundo plano, mientras que -p 8000:8000 conecta el puerto 8000 del computador con el puerto 8000 utilizado por FastAPI dentro del contenedor.

El nombre modelo-creditos-api permite identificar posteriormente el contenedor mediante los comandos de Docker.

Verificación del servicio

Con el contenedor en ejecución, la API puede comprobarse mediante la documentación interactiva de FastAPI:

```
http://127.0.0.1:8000/docs
```

Desde esta interfaz se pueden consultar los endpoints disponibles y realizar pruebas de los servicios de predicción.

También puede verificarse el estado de la API mediante el endpoint:

```
http://127.0.0.1:8000/health
```

El endpoint permite comprobar que el servicio, el modelo y el pipeline de preprocesamiento hayan sido cargados correctamente.

Versionamiento con Git y GitHub

El proyecto utiliza Git para controlar las versiones de los archivos y mantener un historial de los cambios realizados durante el desarrollo. El repositorio se inicializa desde la carpeta raíz del proyecto, manteniendo la estructura de archivos necesaria para que los scripts puedan ejecutarse correctamente.

Flujo de ramas y versiones

La rama principal del repositorio es main y representa la versión integrada del proyecto. Para organizar el desarrollo, los cambios se realizan mediante ramas de trabajo independientes, evitando modificar directamente la versión principal durante cada etapa de desarrollo.

Una vez completados y revisados los cambios de una rama de trabajo, estos se integran a main mediante un proceso de merge. De esta manera, la rama principal mantiene una versión consolidada del proyecto, mientras que las ramas de trabajo permiten desarrollar y probar modificaciones de forma independiente.

El historial de Git permite identificar los cambios realizados en cada etapa mediante commits, facilitando el seguimiento de la evolución del código y la recuperación de versiones anteriores cuando sea necesario.

Estructura y control de archivos

Git permite mantener bajo control de versiones los scripts, archivos de configuración y artefactos que forman parte de la entrega. Al mismo tiempo, se utiliza .gitignore para excluir archivos temporales o generados automáticamente que no deben formar parte del repositorio, como:

```
pycache/  
*.pyc  
.env  
.venv/  
venv/  
*.log  
.streamlit/secrets.toml
```

Los resultados y artefactos que forman parte de la entrega se conservan en el proyecto, incluyendo las carpetas resultados_*, modelo_final.joblib y predicciones_api_monitoring.csv.

Repositorio remoto

El proyecto se sincroniza con un repositorio remoto en GitHub. Esto permite conservar el código y los archivos versionados en un repositorio central, facilitar la entrega del proyecto y mantener un historial de modificaciones.

La estructura interna del proyecto debe conservarse también dentro del repositorio. No se deben cambiar los nombres o ubicaciones de archivos utilizados por los scripts sin actualizar las rutas correspondientes en el código.

El desbalance de clases es una característica central del problema. Por eso el proyecto utiliza `class_weight="balanced"` en Regresión Logística, Random Forest y en la SVM base, y además evalúa las clases por separado.

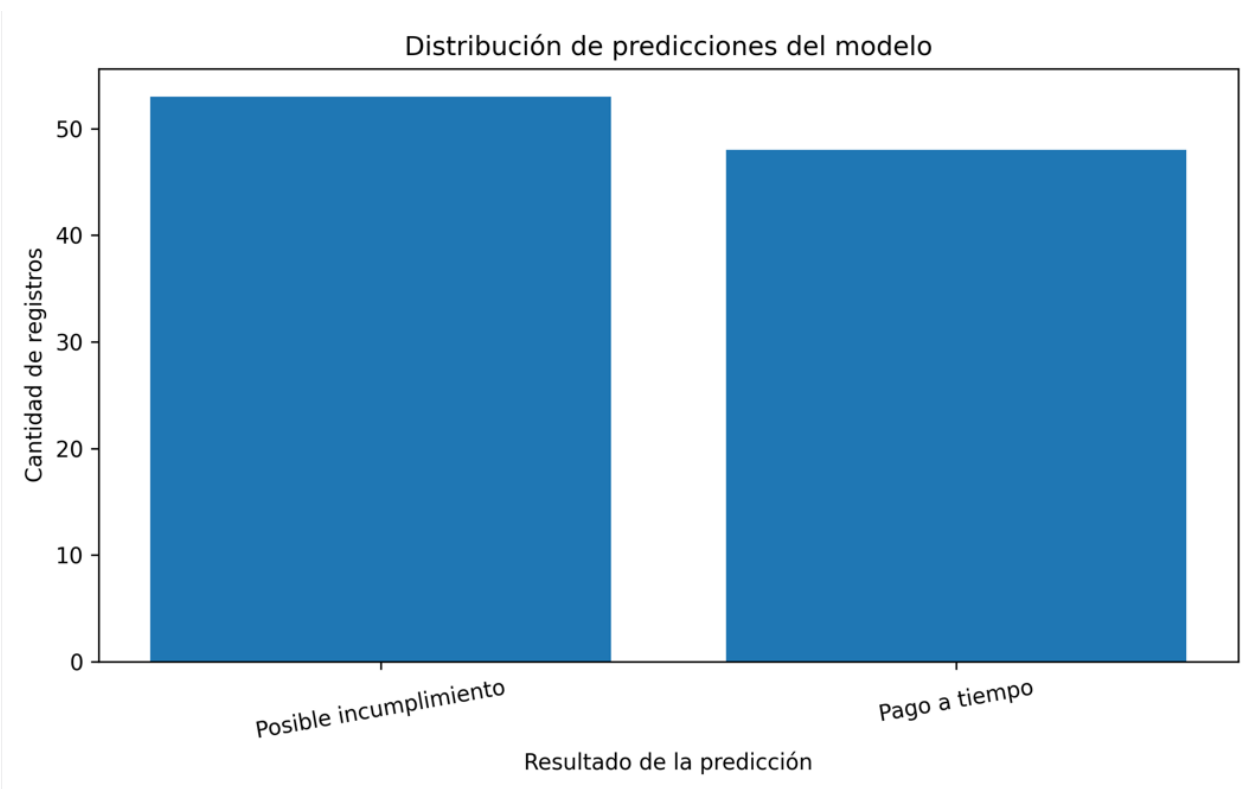
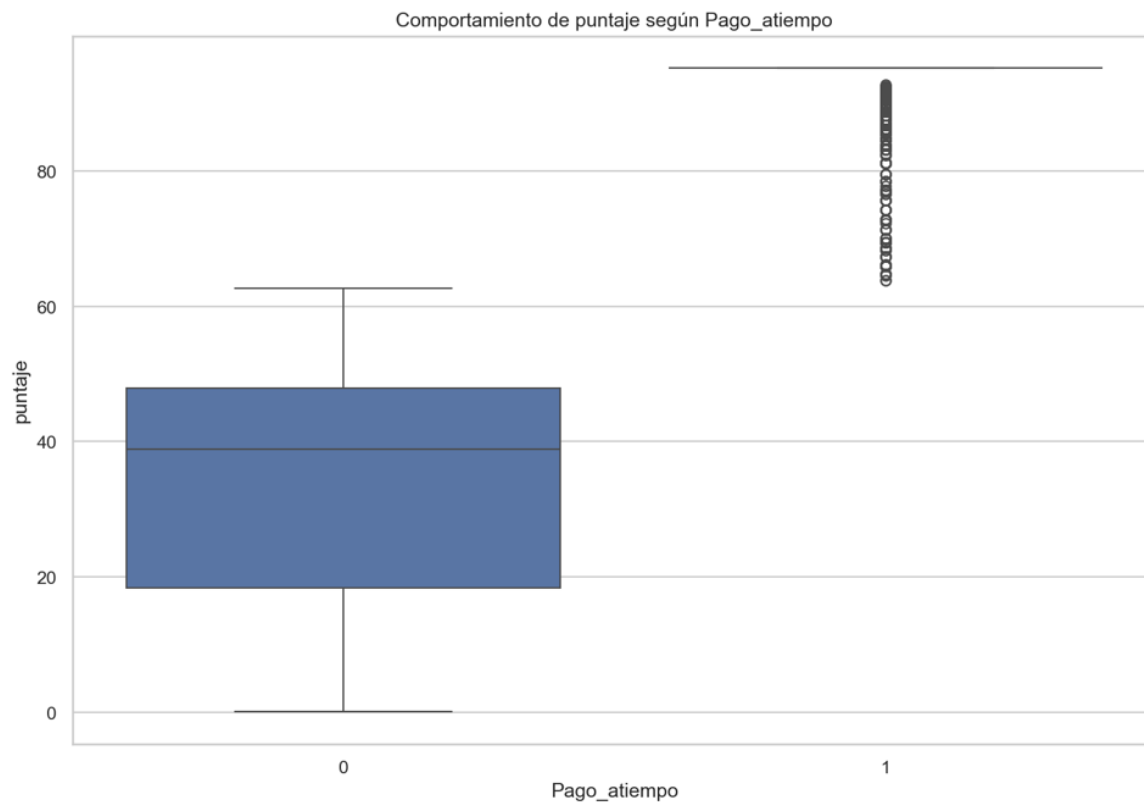
Las probabilidades producidas por el modelo deben interpretarse como salidas del clasificador para la muestra evaluada. El nivel de riesgo es una categorización definida por los umbrales del servicio y no constituye por sí mismo una conclusión causal sobre el comportamiento futuro de una persona.

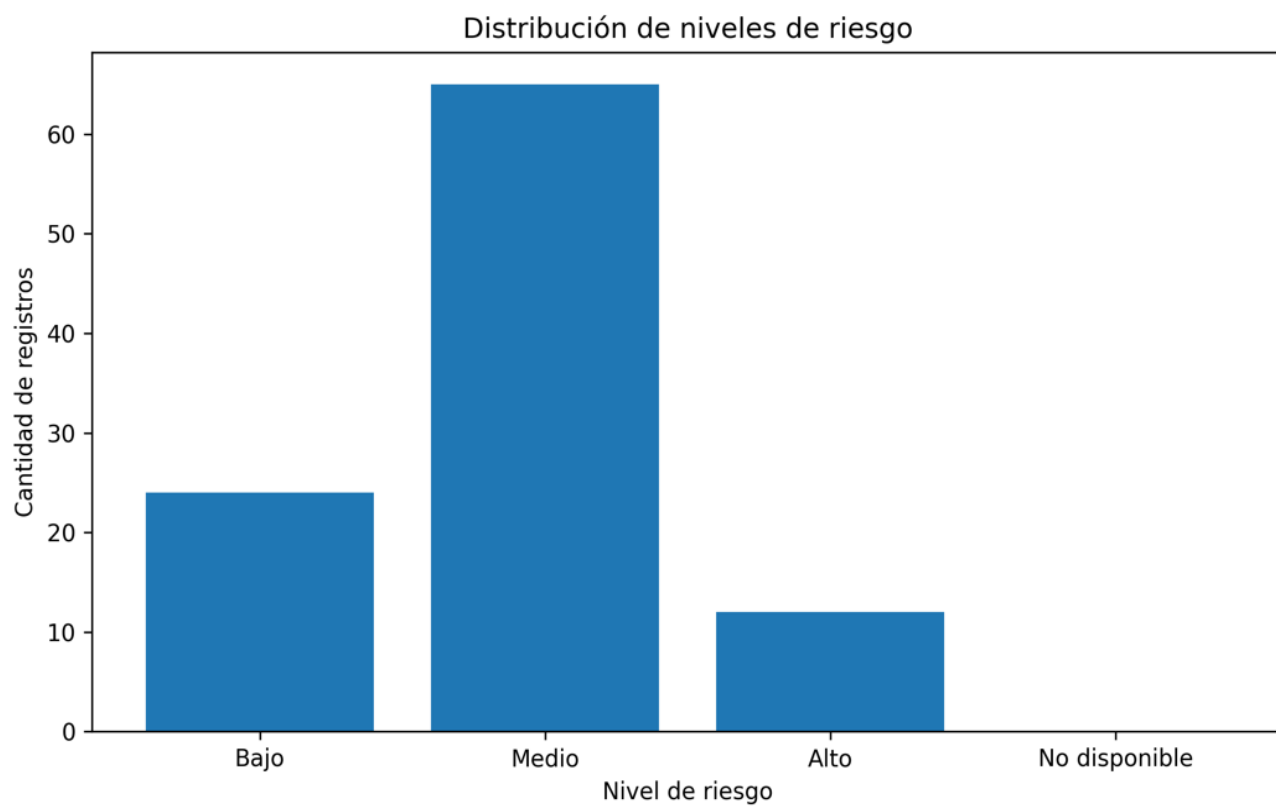
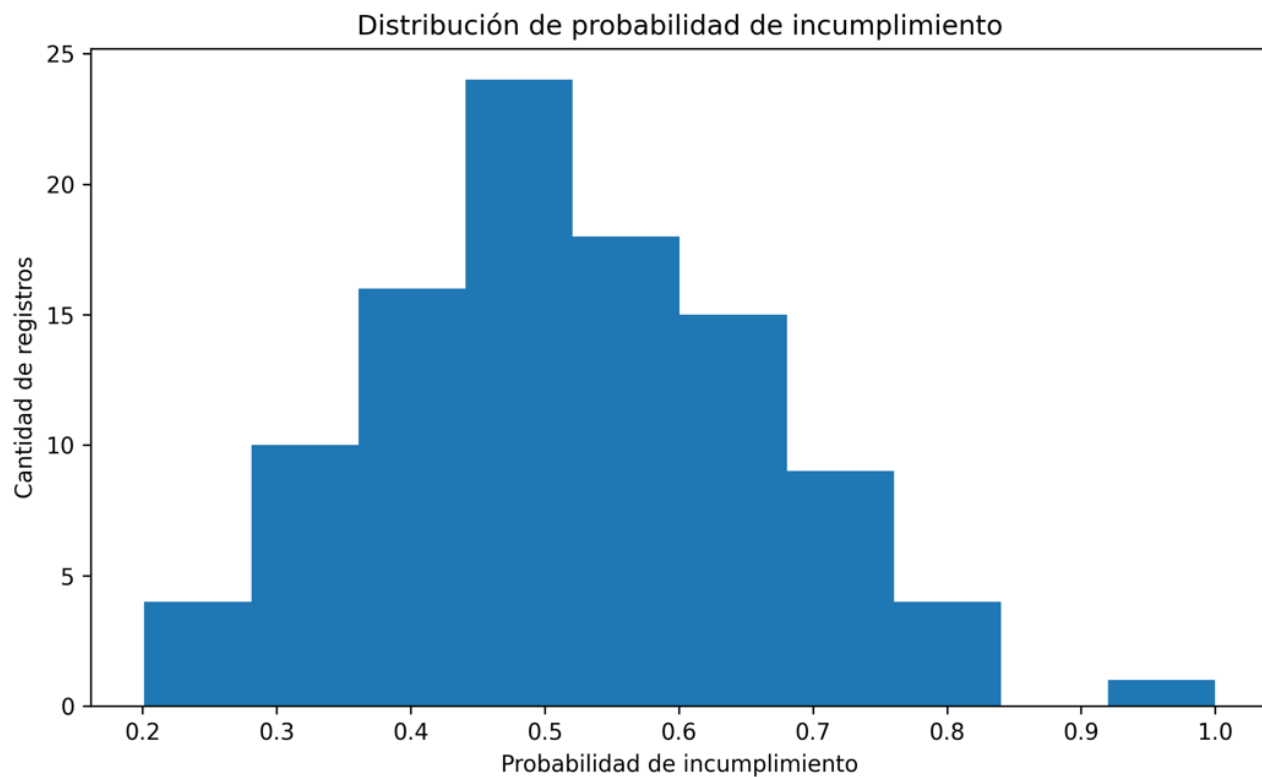
Matriz de correlación de variables numéricas

Variables (X-axis): capital_prestado, plazo_meses, edad_cliente, salario_cliente, total_otros_prestamos, cuota_pactada, puntuaje, puntuaje_datacredito, cant_creditosvigentes, huella_consulta, saldo_mora, saldo_total, saldo_principal, saldo_mora_codeudor, creditos_sectorFinanciero, creditos_sectorCooperativo, creditos_sectorReal, promedio_ingresos_datacredito, Pago_atiempo, anio_prestamo, mes_prestamo, dia_semana_prestamo, hora_prestamo, relacion_cuota_salario, relacion_deuda_salario, saldo_pendiente_estimado.

Variables (Y-axis): capital_prestado, plazo_meses, edad_cliente, salario_cliente, total_otros_prestamos, cuota_pactada, puntuaje, puntuaje_datacredito, cant_creditosvigentes, huella_consulta, saldo_mora, saldo_total, saldo_principal, saldo_mora_codeudor, creditos_sectorFinanciero, creditos_sectorCooperativo, creditos_sectorReal, promedio_ingresos_datacredito, Pago_atiempo, anio_prestamo, mes_prestamo, dia_semana_prestamo, hora_prestamo, relacion_cuota_salario, relacion_deuda_salario, saldo_pendiente_estimado.

Color scale: -0.6 (blue) to 1.0 (red).





API de Predicción de Pago de Créditos

openapi.json

API para predecir si un crédito será pagado a tiempo utilizando un modelo de machine learning.

default

GET	/info
GET	/health Health
POST	/predict Predict
POST	/predict/batch Predict Batch

Schemas

HTTPValidationError	Expand all	object
RegistroCredito	Expand all	object
RespuestaBatch	Expand all	object
ResultadoPrediccion	Expand all	object
SolicitudBatch	Expand all	object
ValidationError	Expand all	object



Monitoreo del Modelo de Crédito

Dashboard de seguimiento del modelo de predicción de incumplimiento de créditos.

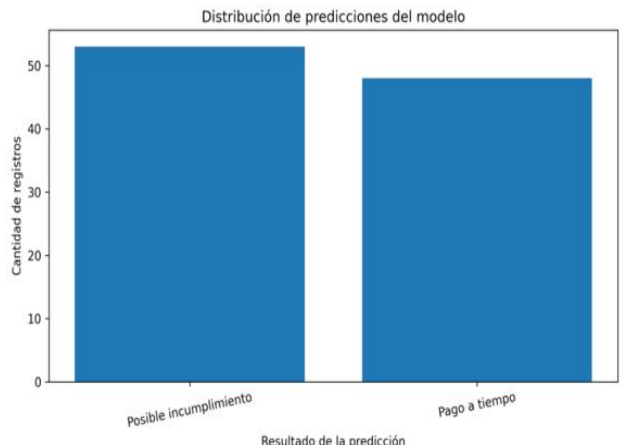
Permite consultar las predicciones generadas por la API, analizar el nivel de riesgo y revisar posibles cambios en la distribución de los datos mediante Data Drift.

Resumen del monitoreo

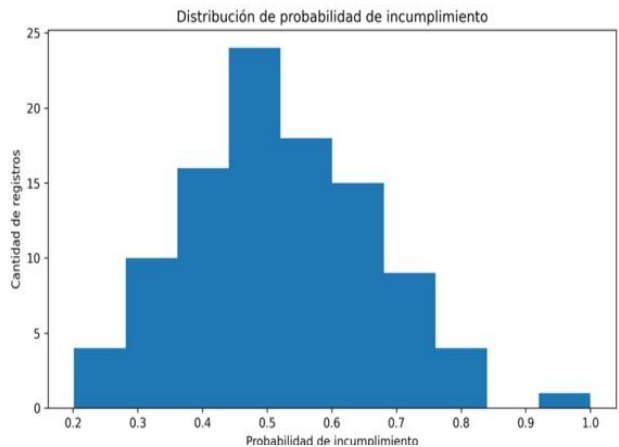
Total predicciones	% Incumplimiento	Probabilidad promedio	Riesgo alto	Tiempo promedio
101	52.48%	51.92%	12	27.21 ms

Distribuciones del modelo

Distribución de predicciones



Probabilidad de incumplimiento



Conclusión

El proyecto construye un flujo completo y trazable de Machine Learning para la predicción de incumplimiento de créditos. El trabajo comienza con una revisión de calidad que transforma la base original en un dataset preparado; continúa con un EDA que documenta la estructura, distribución y relaciones de los datos; y después convierte esa información en características procesables mediante un pipeline reproducible.

La etapa de modelado compara tres enfoques —Regresión Logística, Random Forest y SVM calibrado— utilizando múltiples métricas y validación cruzada. La selección del modelo final prioriza la detección de la clase `No paga a tiempo`, y el modelo seleccionado se persiste para poder reutilizarlo sin repetir el entrenamiento.

A partir de ese artefacto, el proyecto pasa de un experimento de Machine Learning a un proceso de inferencia: deployment aplica el pipeline y el modelo a nuevos registros, FastAPI expone la predicción mediante endpoints, el script de envío simula el ingreso de nuevos datos y monitoring analiza las predicciones y posibles cambios en las distribuciones.

Finalmente, Streamlit reúne los resultados del monitoreo en una interfaz visual. La solución, por tanto, no se limita al entrenamiento de un modelo: integra preparación, análisis, modelado, reutilización, servicio, monitoreo y visualización, manteniendo una relación explícita entre los archivos que producen y consumen cada etapa.